EduAirControl: an IoT-enabled microservices platform for environmental monitoring and analysis of educational spaces

Jhon Jairo Cortez Vargas · María de los Ángeles Olaya García

Technologist in Software Analysis and Development · SENA Industria · Group (Ficha) 3145556

Instructor: Karol Daniela Correa · Neiva, Huila, Colombia · October 10, 2026 · Version 1.0

Abstract

Educational institutions rarely have a single place to check the environmental conditions of their classrooms: temperature, humidity, air quality and noise are usually measured in isolation, or not measured at all. This article presents EduAirControl, a web and mobile platform that centralizes those measurements, links each one to the educational environment and the sensor that produced it, and lets users view them in real time and analyze them by day, week, month and year. The system was built with a microservices architecture in Java 17 and Spring Boot 4, an API Gateway with OAuth2/JWT authentication, PostgreSQL persistence managed with Liquibase, RabbitMQ messaging, Redis caching, a React frontend and an Expo/React Native mobile app. A prototype based on an ESP32 with DHT11, KY-038 and MQ-135 sensors validates physical data capture. The project includes seven backend modules with 285 automated test methods and an eight-level test plan. Limitations are documented as well: the final firmware that sends readings to the API and the MQ-135 calibration are still pending, and the full execution of the test plan has not yet been closed.

Keywords: environmental monitoring, Internet of Things, microservices, Spring Boot, ESP32, educational spaces, hexagonal architecture.

1. Introduction

The environmental conditions of a classroom affect the comfort and attention of the people who use it. Even so, the data produced by sensors tends to be scattered: each device shows its own value, nobody keeps the history, and there is no clear link between a measurement and the space where it was taken. Whoever manages the spaces cannot compare how conditions change over time or make decisions based on data.

EduAirControl was created to close that gap. Its value proposition rests on four pillars defined in the product vision: **centralization** of information in a single platform, **context** (each measurement tied to an environment, a sensor and a time), **understanding** through dashboards and historical analysis, and **protection** of access through registration, authentication, roles and auditing. The product does not promise automatic physical control or predictive management; those capabilities are outside the current scope.

2. Problem statement

Users and administrators of an institution need to understand the environmental state of their spaces, but sensor information is neither centralized nor analyzed. Five dimensions follow from this:

Dimension	Problem	Product implication
Access	There is no clear place to consult the information	Authenticated dashboard
Centralization	Measurements stay detached from the space	Management of environments, sensors and measurements
Understanding	Raw values do not explain how conditions evolve	Analysis by day, week, month and year
Traceability	A measurement is only useful if its origin and time are known	Sensor–environment–timestamp relationship
Governance	Access and administration must be controlled	Roles, permissions, sessions and auditing

3. Objectives

General objective. To develop a platform that makes it possible to monitor, manage and analyze the environmental conditions of educational spaces using IoT sensor data.

Specific objectives.

- Implement authentication and authorization with roles (SUPER_ADMIN, ADMIN, USER and VIEWER).
- Manage campuses, educational environments, sensors and devices.
- Receive, validate, store and analyze measurements, and raise alerts based on thresholds.
- Provide web and mobile interfaces with a dashboard, favorites, preferences, languages and accessibility.
- Validate physical data capture with an ESP32-based prototype.
- Document and test the system so that its results can be verified.

4. Methodology

The team followed specification-driven development (SDD): context, domain, product and requirements were documented first, followed by architecture, data and API contracts. Documentation was organized into folders for governance, context, domain, product, requirements, architecture, data, API, UML, microservices, DevOps, quality, operations and project control.

Design decisions were recorded as ADRs (architecture decision records). Among them, each service is an independent project with its own dedicated database schema and follows a **hexagonal architecture**, which separates the domain from inbound and outbound adapters. Service contracts are described in OpenAPI. Quality relied on automated tests, an end-to-end smoke test and a continuous integration pipeline; containerization was done with Docker Compose, and the path toward Kubernetes was prepared.

5. Architecture and implemented solution

5.1 Overview

The system follows a client–server model. The web and mobile applications consume an HTTP/JSON API exposed by an **API Gateway** (Spring Cloud Gateway, port 8080) that routes requests, validates the JWT, applies rate limiting and propagates a correlation ID. Six microservices sit behind the gateway:

Service	Responsibility	Port
ms-security	Registration, login and logout, OAuth2/OIDC, RS256 JWT, roles and institutions	8081
ms-classroom-management	Campuses, environment types and educational environments	3002
ms-environment-monitoring	Measurements, dashboard, analysis, thresholds and alerts	3003
ms-sensor-management	Catalog, sensors, installations, variables and devices	3004
ms-user-experience	Favorites, preferences and ratings	3006
ms-user-management	User profile	3007

5.2 Technology stack

Layer	Technology
Backend	Java 17, Spring Boot 4.0.5, Spring Data JPA
Security	Spring Authorization Server, RS256-signed JWT, Google and Facebook sign-in (authorization code flow)
Data	PostgreSQL 15, Liquibase (`ddl-auto=validate`)
Messaging and cache	RabbitMQ 3.13, Redis 7 (token revocation list)
Web	React 19, Vite 8, React Router, i18next, Recharts
Mobile	Expo 54, React Native 0.81, AsyncStorage, BLE PLX
Containers and CI	Docker Compose, continuous integration pipeline, Kubernetes manifests

5.3 Environmental information flow

1. Sensors capture the variables of the educational space.
2. The ESP32 processes the readings and, in the final integration, sends them to the ingestion endpoint.
3. The backend validates the device key (`X-API-Key`), processes the measurement and stores it in PostgreSQL; events travel through RabbitMQ.
4. The web and mobile apps query the API and display measurements, statuses, analyses and alerts.

5.4 Security

User access is based on JWT with expiration and revocation (on logout the token is recorded in Redis and any reuse is rejected with a 401). Roles separate administrative operations from read-only ones. Devices use an `X-API-Key` that is shown only once when it is created or rotated. Secrets are configured through environment variables and CORS is configurable.

5.5 IoT prototype

Component	Role in the prototype
ESP32	Controller and Wi-Fi communication; BLE provisioning from the mobile app
DHT11	Air temperature and humidity
KY-038	Relative sound level (analog output)
MQ-135	Relative variation in gases and air quality

The MQ-135 needs calibration and voltage adaptation before the ESP32's analog-to-digital converter, so it is **not presented as a calibrated CO_2 meter in ppm**. The delivered files contain DHT11 and KY-038 tests at 115,200 baud.

6. Quality and testing

The test plan (version 1.0) defines eight levels: unit, integration, contract, functional, system, IoT, regression and non-functional. It includes a matrix of 60 functional cases (security, environments, sensors, monitoring, web, mobile and IoT), entry and exit criteria, a result classification (PASS, FAIL, BLOCKED, NOT RUN, NOT APPLICABLE) and defect management.

The automated inventory taken from the source code shows this distribution:

Service	Test methods
ms-environment-monitoring	83
ms-security	57
ms-sensor-management	55
ms-classroom-management	33
API Gateway	27
ms-user-management	20
ms-user-experience	10
Total	**285**

These tests cover application services, controllers, JWT, the Redis revocation list, rate limiting, event consumers and the domain rules for alerts and analysis.

Execution status. The execution record dated October 9, 2026 is a draft: every case is NOT RUN, except IOT-008 (BLOCKED because the sending firmware does not exist yet). As background, a report from October 7 shows 10 of 10 tests passing in ms-user-experience and, in ms-environment-monitoring, 28 passing and 58 with errors. That error comes from a duplicated bean caused by an outdated compiled class in `target/`, which a clean build probably resolves; if it persists, it will be handled as a defect. These figures are not presented as the final outcome of the test round.

7. Results

- The outcome is a platform with seven backend modules (gateway plus six microservices), a web application and a mobile application.
- All eight capabilities of the minimum scope are covered (authentication, environments, sensors, monitoring, dashboard, historical analysis, favorites and environment ratings).
- A complete documentation set was delivered: requirements specification, analysis, technical proposal, design, database, test plan, quality, implementation, and technical, installation, administrator and user manuals.
- The physical prototype demonstrates temperature, humidity and sound readings, and BLE provisioning.

Limitations. (1) The final firmware that sends measurements to the API does not exist yet. (2) The MQ-135 is not calibrated. (3) There are no automated frontend tests. (4) Load testing, certified penetration testing and disaster recovery were not carried out. (5) The test plan still has to be executed in full and reviewed by the technical and functional owners.

8. Conclusions

EduAirControl shows that it is feasible to centralize the environmental data of educational spaces in a secure, traceable and scalable platform, using hexagonal microservices and a low-cost IoT prototype. Splitting the system into services, OpenAPI contracts and Liquibase change management make it easier to evolve the system without breaking what already exists.

The main lesson of the process is that documenting and testing honestly is as valuable as building: separating what has been executed from what is pending makes results verifiable and turns limitations into conscious decisions rather than surprises.

Future work. Complete the ingestion firmware with Wi-Fi reconnection; calibrate the MQ-135 or replace it with a certified CO_2 sensor; execute and close the test plan; add automated web and mobile tests; run load and security tests; and deploy to a staging environment to validate rollback.

References

1 EduAirControl. *Technical Manual*, version 1.1. SENA Industria, 2026.
2 EduAirControl. *Installation Manual*. SENA Industria, 2026.
3 EduAirControl. *Test Plan*, version 1.0, October 9, 2026.
4 EduAirControl. *Product, architecture and requirements documentation* (folders 01 to 15 of the documentation repository), 2026.
5 Cockburn, A. *Hexagonal Architecture* (Ports and Adapters), 2005.
6 VMware/Broadcom. *Spring Boot*, *Spring Cloud Gateway* and *Spring Authorization Server*: official documentation.
7 PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*.
8 Espressif Systems. *ESP32 Series Datasheet*.